Memory as Governed Infrastructure: An Ontology-Grounded, Scope-Hierarchical Memory System for Enterprise Agents

MLPal Research*

2026

Abstract

Memory systems for LLM agents have converged on a common recipe—extract facts from interaction streams, store them in a vector or graph index, retrieve them at inference time—while treating tenancy, governance, temporal correctness, and read-path economics as afterthoughts. We argue these are not afterthoughts but the binding constraints for *enterprise* memory, where a single store serves many organizations, memory crosses trust boundaries between individuals and their employers, facts change and conflict, and every agent turn issues a read. We present MLPal Memory, a production memory system organized around five commitments: (1) *governance in the write path*—consent, extraction policy, and secret redaction are pipeline stages that run before persistence, not API middleware; (2) a *subject-scope hierarchy*—memory compounds per user, team, organization, repository, service, and agent, and reads resolve across the caller's accessible scopes with narrowest-wins semantics and full provenance; (3) *bi-temporal writes*—facts are superseded, never destroyed, supporting as-of reads and auditable correction; (4) a *two-tier store* that separates verbatim, citeable passages from derived, ontology-typed facts carrying evidence links; and (5) a *deterministic read path*—hybrid lexical/vector/graph retrieval with reciprocal-rank fusion that issues zero LLM calls per query. We compare MLPal Memory against six contemporary memory systems (Mem0, Zep/Graphiti, Letta, Cognee, LangGraph memory, and a managed cloud offering) along structural, temporal, tenancy, governance, and cost dimensions, grounding the comparison in source-level analysis of the systems' shipped code and published documentation rather than self-reported benchmarks. We find that no surveyed system combines scope-hierarchical tenancy with write-path governance, and that several place one or more LLM calls on the retrieval path—a cost structure we argue is incompatible with memory-on-every-turn deployment. We evaluate the deployed system on one organization's real working corpus: retrieval quality measured as a stored correction trajectory against grep, naive-RAG, and FTS baselines on identical data; maintenance contracts executed as live probes against the deployment (catching two deployment-grade bugs CI had missed); an agent-in-the-loop A/B whose efficiency null we report in full; and an escalation study locating the boundary of what memory transmits — knowledge, not capability.

1 Introduction

Long-term memory has become production infrastructure for LLM agents. Systems such as Mem0 [5], Zep/Graphiti [21, 29], Letta/MemGPT [19, 16], Cognee [25], and LangGraph memory [15] continuously ingest interaction streams, extract structured facts, and serve them back to agents at inference time; hyperscalers now ship managed equivalents [2]. The research community has kept pace with architectures for dynamic memory organization [27], associative graph memory [11, 12], reflective consolidation [23], and a growing suite of benchmarks [17, 26, 22].

Nearly all of this work optimizes a single figure of merit: answer quality on conversational recall for a *single principal*—one user, one assistant, one trust domain. Enterprise deployment breaks that frame in four ways.

Memory crosses trust boundaries. A company-wide memory records what individuals said and did. Some of that knowledge belongs to the individual (a personal preference stated in a chat), some to a team (an

*Systems paper accompanying two companion papers on memory fidelity measurement and sound selective repair [4, 3]. Code, evaluation harnesses, and raw results: the `mlpal-memory` repository.

incident postmortem), and some to the organization (a deprecation decision). A store that cannot distinguish these—and enforce the distinction on both write and read—either leaks personal context to colleagues or starves shared memory to avoid doing so. Surveyed systems offer flat per-user stores or flat dataset ACLs; none models the subject hierarchy that organizational knowledge actually has (section 5).

Governance must precede persistence. Consent, per-source extraction policy, and secret hygiene are commonly implemented as API-layer concerns, if at all. But an extraction pipeline that has already persisted a credential, or derived a fact from a conversation whose author opted out, cannot un-know it cheaply: derived artifacts propagate through summaries, embeddings, and indexes. Our companion measurement study finds that no widely deployed memory system ships a testable deletion or maintenance contract [4]. We argue the only robust placement for governance is *inside* the fold pipeline, ahead of extraction and persistence (section 3).

Facts change, conflict, and must remain auditable. Organizational truth is revised (a service is re-owned), contradicted (two engineers disagree), and regulated (what did we believe when we made that decision?). This requires bi-temporal storage—separating validity time from ingestion time—with supersession rather than destruction, a property pioneered for agent memory by Graphiti [21] but absent from most production systems we survey, several of which model updates as delete-and-replace or timestamp tagging (section 5).

Reads are the hot path. A memory consulted on every agent turn has a read:write ratio far above one. Placing an LLM call on the retrieval path—as several surveyed systems do, in one case up to thirteen calls per query in the flagship configuration—couples per-turn latency and cost to model inference. We take the opposite position: retrieval must be deterministic and LLM-free, with model capacity spent at write time, where it can be tiered, batched, and deferred (section 6).

Contributions

We present MLPal Memory, a deployed memory system built around these constraints, and make four contributions:

1. **A design framework** for enterprise agent memory: five commitments (write-path governance; subject-scope hierarchy; bi-temporal supersession; two-tier direct/derived storage with provenance; deterministic reads) and the argument for why each is load-bearing rather than optional (section 3).

2. **A system realization** of the framework: a generic episode envelope and source-plugin ingestion model; a single governed fold pipeline (consent → policy → redaction → direct store → tiered extraction → resolution → bi-temporal write); ontology-closed extraction; and cross-scope resolution with explainable provenance, implemented on a single relational store with vector and lexical indexes (section 4).

3. **A source-grounded comparison** of six contemporary memory systems along structural, temporal, tenancy, governance, integration-surface, and read-cost dimensions. Unlike benchmark self-reports—whose reliability is contested [7, 10]—our comparison rests on reading the systems' shipped code (for the open-source systems) and primary documentation (for the closed ones), as of July 2026 (section 5).

4. **An execution-grounded evaluation.** We evaluate the deployed system on a real organizational corpus: a stored, reproducible retrieval-quality trajectory against a grep baseline (including the configurations that lost, and measured single-leg baselines), live verification probes that run the maintenance contracts against the deployment (two of which caught deployment-grade bugs), an agent-in-the-loop A/B whose efficiency null we report in full, and an escalation study that locates the boundary of what memory can transmit (section 7).

MLPal Memory is not a claim that retrieval-quality research is solved; it is a claim that the systems layer beneath it has been under-specified, and that governance, tenancy, temporal semantics, and cost structure determine whether a memory system is deployable at all.

2 Related Work

Production memory systems. MemGPT introduced OS-style virtual context management with agent-editable memory tiers [19]; Letta extends this line with git-backed "context repositories" in which agents version file-based memory and merge concurrent edits [16]. Mem0 extracts natural-language facts per interaction and applies ADD/UPDATE/DELETE/NOOP operations against a vector store, with an optional graph variant [5]. Zep's Graphiti engine builds a bi-temporal knowledge graph with hybrid semantic/BM25/graph retrieval and invalidate-not-delete conflict handling [21, 29]; its bi-temporal model is the closest prior art to our temporal design. Cognee organizes ingestion as Extract–Cognify–Load pipelines over pluggable graph, vector, and relational backends, with seventeen retrieval modes and a usage-driven feedback-weighting loop [25, 6]. LangGraph provides namespaced JSON document stores with semantic/episodic/procedural typing [15]; AWS AgentCore Memory offers managed short/long-term memory [2]. Our comparison (section 5) differs from vendor surveys in being grounded in source-level analysis of shipped code rather than reported capabilities.

Memory architectures in research. A-MEM organizes memories as an evolving Zettelkasten with LLM-generated links [27]. HippoRAG and its successor apply Personalized PageRank over extraction graphs to approximate associative memory [11, 12]. Reflective Memory Management combines prospective topic decomposition with retrospective, feedback-trained reranking [23]; generative-agent memory streams introduced importance/recency/relevance scoring and periodic reflection [20]. These lines are complementary to ours: they refine *what to store and how to rank it*, whereas we specify the tenancy, governance, and temporal substrate any of them would need to run inside an enterprise.

Benchmarks and their limits. LoCoMo [17], LongMemEval [26], and MemBench [22] evaluate long-horizon conversational memory; HotpotQA and successors evaluate multi-hop composition [28]. Vendor results on these suites are contested: LoCoMo scores for the same systems differ by tens of points across vendor reproductions, and LLM-as-judge protocols are sensitive to rating indeterminacy [10]. Our companion work argues the deeper problem is that paraphrase-tolerant judging is structurally blind to write-time information loss, and contributes a fidelity workload with programmatic ground truth—exact-quote, attribution, as-of, cross-source join, supersedence, and deletion-audit query classes—over a multi-source work corpus [4]. A second companion develops the maintenance side: a guard algebra under which *selective repair* of derived memory after an upstream correction is provably equivalent to a clean rebuild at a fraction of the cost [3]. MLPal Memory is the system these instruments were designed to hold to account; we adopt both as our evaluation contract (section 7).

Multi-tenant data systems. Our isolation design draws on standard practice for shared-database multi-tenancy—application-enforced tenant predicates at a single choke point, with row-level security as defense-in-depth—and our lifecycle roadmap draws on versioned data systems (commit/branch/merge semantics over facts rather than text) [24]. Knowledge-conflict handling in collaborative settings—survival-based reputation [1], contradiction benchmarks [13]—informs the conflict-ranking layer we sketch as future work.

3 Design Commitments

MLPal Memory is organized around five commitments. Each is stated with the failure mode it prevents; section 4 describes their realization.

3.1 C1: Governance runs in the write path

Consent state, extraction policy, and secret redaction execute as stages of the fold pipeline, *before* extraction and persistence. A denied episode is recorded as processed-with-reason and produces no memory; a secret is replaced by an opaque vault reference before any model sees the content.

Why not middleware? Post-hoc governance must claw back derived artifacts: facts, embeddings, summaries, and indexes computed from content that should never have entered the store. Our companion study shows this claw-back is exactly where deployed systems have no testable contract—one vendor's documentation states that deleting an episode leaves facts derived from it visible unless separately invalidated [4].

Running governance first makes the expensive problem (provable forgetting of derived state) small, because ungoverned content never generates derived state.

Policy resolution is *deny-precedence* down the scope chain: an organization's deny rule overrides a user's allow; consent is tri-state (active / paused / cleared, where clearing purges). Redaction pushes matched secrets (key–value assignments, cloud and VCS token formats, PEM blocks) into a secret-vault seam and substitutes stable references, so memory can cite *that* a credential was exchanged without holding it.

3.2 C2: Memory compounds per subject, and reads resolve a scope set

Every episode and every derived fact carries a scope reference (σ, s), where σ is one of global, org, team, service, repo, agent, or user, and s identifies the subject. org and global are backdrops; the remainder are *subjects* that accumulate their own memory: a repository has conventions, an agent has operating experience, a user has preferences. A retrieval context activates subjects ("user u, in repository r, using agent a, at organization o") and the resolver takes the union of accessible scopes, deduplicates by typed key, prefers the narrowest scope on conflict, and reports shadowed alternatives as provenance. An `explain` endpoint returns the full resolution trace.

Why a hierarchy rather than per-user stores or flat ACLs? Per-user stores cannot represent shared knowledge without copying it (and copies then drift); flat dataset ACLs can share but cannot *layer*—they have no notion that a team's convention overrides the organization's default, or that a personal preference overrides both, which is precisely how organizational knowledge behaves. The hierarchy also carries the privacy invariant: user-scoped memory is owner-only, enforced at read time for every principal including administrators.

3.3 C3: Bi-temporal supersession, never destruction

Edges carry validity intervals (`valid_at/invalid_at`) and system intervals (`ingested_at/expired_at`). New information that contradicts a functional relation closes the old edge's validity at the new edge's start; explicit end events close open edges; nothing is deleted by revision. Reads accept an `as_of` timestamp in either timeline; node visibility follows edge validity, so a point-in-time query reconstructs the graph as it was believed—or as the world was—at that instant.

Why bi-temporal rather than timestamps? Single-timeline stores conflate "when we learned it" with "when it was true," making both audit questions (what did we believe when we decided?) and correction questions (what was actually true?) unanswerable. This design follows Graphiti's temporal model [21]; we differ in composing it with scope partitioning and in exposing as-of semantics through the same governed read path as current-time queries. The only sanctioned forgetting is governance-initiated purge (consent clear, retention policy), which is thereby an auditable event rather than a side effect of update—the property our companion fidelity workload tests as *deletion audit* [4].

3.4 C4: Two tiers—verbatim evidence and derived belief—linked by provenance

The *direct* tier stores content verbatim: documents and conversations, chunked, embedded, and citeable. The *derived* tier stores ontology-typed nodes and edges extracted from that content, each carrying confidence and `derived_from` links to its source episodes and chunks. Search returns both tiers, labeled by origin.

Why two tiers? Derived facts are lossy by construction—that is their value—but agents and auditors need the loss to be recoverable: a fact that cannot produce its evidence is an unverifiable assertion, and our companion measurements show exact-quote and attribution queries are precisely where extraction-only stores fail silently [4]. Conversely, verbatim-only stores (classic RAG) push all structure recovery to query time. Storing both, with explicit provenance edges, lets the read path fuse them and lets governance operate on the closure: purging a source can enumerate its derived descendants—the precondition for the sound selective repair developed in companion work [3].

3.5 C5: Extraction is ontology-closed; reads are deterministic

A small, versioned core ontology—entity classes, edge types, and an action vocabulary—bounds what may enter the derived tier. The deterministic extractor maps recognized action types to typed subgraphs; the LLM extractor (engaged only for content-bearing episodes) emits into a closed JSON schema and must quote a verbatim evidence span from the (already-redacted) content, with non-conforming facts dropped. Entity resolution upserts authoritative types on exact $(org, scope, type, key)$ and deduplicates semantic entities by embedding similarity within type and scope.

The read path is a fixed, deterministic program: parallel lexical and vector candidate generation under a mandatory scope predicate, reciprocal-rank fusion [8], a graph-distance boost from the caller's anchor nodes, as-of filtering, and cross-scope resolution. It issues *zero* LLM calls.

Why closed extraction and LLM-free reads? An open extraction vocabulary degenerates into untyped soup that defeats resolution, supersession (which needs to know a relation is functional), and policy (which names types). And every LLM call on the read path multiplies per-turn cost and tail latency by model inference; section 6 quantifies the gap against surveyed systems. Model capacity belongs at write time, where it can be tiered by salience, batched, and deferred—and where its outputs are checked against schema and evidence.

4 System Architecture

MLPal Memory is a single service (Python/FastAPI) over a shared relational store (PostgreSQL with `pgvector` [14]); a SQLite profile runs the identical code path for offline development and testing. We describe the flow of an event from source to answer.

4.1 Ingestion: episode envelopes and source plugins

All capture converges on one *episode envelope*: an event identifier, actor, action type from the ontology's vocabulary, subject references, optional content, and a target scope (σ, s). Producers are untouched: a *source plugin* adapts an existing system of record (audit logs, agent run stores, VCS events, chat transcripts) into envelopes, and a capability-driven scheduler tails pull sources with per-source keyset watermarks, giving idempotent, resumable ingestion. Push ingestion (`POST /episodes`) authenticates producers and pins tenant identity server-side; only service principals may assert an organization, and non-admin writers are confined to their own `user` scope and `org`. Episodes are append-only and deduplicated by event identifier; capture is fire-and-forget and never blocks the producing system.

4.2 The fold pipeline

A background worker (advisory-locked, so replicas do not double-fold) processes each episode through one pipeline:

1. **Tier routing.** Content-bearing action types route to the LLM extraction tier; telemetry-grade events take the deterministic tier. This is the primary cost lever: high-frequency, low-salience traffic never touches a model.
2. **Gates (C1).** Consent is checked down the scope chain, then deny-precedence extraction policy over source, action type, and metadata. A refusal marks the episode processed with a machine-readable drop reason—degradation is distinguishable from success.
3. **Redaction (C1).** Secrets are vaulted and replaced by references before any storage or extraction.
4. **Direct tier (C4).** Content is stored verbatim as a document, chunked by paragraph, and embedded.
5. **Extraction (C5).** Deterministic rules always run; the LLM extractor runs only in the LLM tier, emits into the closed ontology schema, and drops facts whose evidence span is not verbatim in the redacted content.

6. **Resolution (C5).** Entities resolve by exact authoritative key or embedding similarity within type and scope.

7. **Bi-temporal write (C3).** Edges are upserted with validity stamped from the episode's event time; functional-relation contradictions deterministically supersede; non-functional contradictions in the LLM tier are adjudicated by a judge constrained to invalidate only the older of two temporally overlapping claims. Every node and edge is stamped with scope, classification, owner, and `derived_from` provenance.

4.3 Storage and the isolation choke point

Nodes, edges, episodes, documents, and chunks live in one schema, partitioned logically by tenant and scope. Every read query composes a single shared predicate—the *scope clause*—which requires an explicit list of accessible (σ, s) pairs; an empty list yields the empty result, never a tenant-wide scan. The clause is the audited choke point: isolation holds by construction of the one function every query path must call, backed by a permanent cross-scope leak test, with database row-level security available as defense-in-depth. global scope is the sole deliberate cross-tenant surface: platform-curated, near-empty, and write-locked.

The graph layer is a driver interface (14 operations, every read taking the scope set) with the relational implementation as default; the interface preserves reversibility toward dedicated graph backends without exposing their query languages to callers.

4.4 Retrieval

A query fans out over (a) an approximate-nearest-neighbor search over node embeddings (HNSW, with iterative scanning to preserve recall under scope filtering), (b) a lexical leg combining full-text rank, trigram similarity, and key prefix match against expression indexes, and—for the direct tier—(c) chunk retrieval over the same scope set. Legs fuse by reciprocal rank [8]; a graph-distance boost favors results near the caller's anchor nodes (their user node, their active agents) via a depth-bounded traversal; as-of constraints apply the bi-temporal clauses; and the cross-scope resolver (C2) merges, shadows, and annotates. The `explain` variant returns scopes considered, per-scope hits, and what shadowed what. No stage invokes a model.

4.5 Access control and the agent surface

Identity arrives as platform-issued tokens (RBAC permissions plus attribute checks); team membership derives from explicit grants; user-scope reads require ownership, with no administrator bypass. Agents integrate through a Model Context Protocol (MCP) sidecar that is deliberately *read-only*, forwards the caller's own bearer token rather than holding a service credential, refuses service tokens presented as caller tokens, and holds no secrets—the REST API remains the single enforcement boundary. The contrast with agent surfaces that expose unauthenticated write and wipe operations is examined in section 5.

4.6 Operational envelope

The service deploys as a standard replicated Kubernetes workload with horizontal autoscaling; migrations are additive; the entire behavioral test suite runs offline on SQLite with deterministic embeddings, and a second suite runs the vector, lexical, temporal, and row-level-security paths against migrated PostgreSQL in CI (section 7).

5 Source-Grounded Comparison

We compare MLPal Memory to six systems: Mem0, Zep/Graphiti, Letta, Cognee, LangGraph long-term memory, and AWS AgentCore Memory. For the open-source systems the comparison is grounded in reading the shipped code at pinned versions (Cognee v1.4.0, July 2026; Graphiti, Mem0, and Letta at their July 2026 public heads) and primary documentation; for managed services we rely on vendor documentation. We deliberately do not rank systems by self-reported benchmark scores: LoCoMo results for identical systems

Table 1: Capability comparison, July 2026. *Structure*: primary derived representation. *Temporal*: **bi-t** = bi-temporal validity + supersession; **ts** = creation timestamps / soft invalidation; **event** = event-time tagging of extracted events; **none** = no temporal semantics. *Tenancy*: memory-partition model. *Gov.*: write-path governance (consent gates, extraction policy, content redaction). *Prov.*: derived facts link to verbatim source. *Read LLM*: minimum LLM calls on the default useful retrieval path (section 6).

System	Structure	Temporal	Tenancy	Gov.	Prov.	Read LLM
MLPal Memory	ontology-typed graph + passages	bi-t	7-scope hierarchy	✓	✓	0
Zep/Graphiti	typed temporal graph	bi-t	per-graph namespaces	–	✓	0
Mem0	NL facts (+opt. graph)	ts	per-user collections	–	partial	0
Letta	files + blocks (git-backed)	none	per-agent repos	–	partial	0[a]
Cognee v1.4	entity graph + chunks	event	tenant→dataset ACLs	–	✓	1–13
LangGraph	namespaced JSON docs	none	namespace keys	–	–	0
AgentCore	managed extraction store	ts	per-actor/session	–	n/a	n/a

[a]Letta's retrieval is agent-driven: the agent itself spends turns searching memory, so cost appears in the agent loop rather than the store.

differ by tens of points across vendor reproductions [17, 7], judge protocols are sensitive to indeterminacy [10], and our own reading of one vendor's published headline (a 0.93 vs. 0.4 correctness claim) found the two numbers come from different metrics on a 24-question subset, with the weaker baseline absent from the released artifacts [6]. Capability claims, unlike scores, can be checked against source.

5.1 Structure and temporal semantics

Graphiti is the only surveyed system sharing our bi-temporal commitment (validity and transaction timelines, invalidate-not-delete) [21]; we compose that model with scope partitioning and governed reads. Mem0 records creation timestamps and soft-invalidates on update [5]. Letta's file-based memory has no temporal semantics beyond git history [16]. Cognee, despite a temporal search mode, is not bi-temporal: its pipeline extracts *event* nodes with occurrence times and range-filters them; facts carry no validity intervals, contradiction produces coexisting assertions, and the sanctioned update path is delete-and-replace of the source item (`update()` = delete → add → re-cognify). As-of reads—"what did the store believe at t?"—are expressible only in MLPal Memory and Graphiti among the surveyed systems.

5.2 Tenancy and privacy

No surveyed system models a subject hierarchy. Mem0 and AgentCore partition per user/actor; Letta per agent; LangGraph by caller-chosen namespace; Graphiti by graph namespace (Zep's managed layer adds per-user graphs). Cognee ships the most developed open-source tenancy—backend-enforced dataset ACLs (read/write/delete/share) over a flat tenant→dataset model, with physical database-per-dataset isolation—but has one level, no attribute-based checks, and no personal-memory tier: privacy is emergent from not sharing a dataset. MLPal Memory's layered resolution (narrowest-wins with shadowing provenance) and owner-only `user` scope, enforced against administrators, have no counterpart in the surveyed systems. Physical DB-per-dataset isolation is also an operational scaling liability at organizational dataset counts; MLPal Memory instead audits one predicate over a shared store, accepting that the choke point must be defended by construction and test.

5.3 Governance

Here the field is thinnest. Across the surveyed systems we find no consent model, no extraction-policy mechanism, and no ingestion-time content redaction: Cognee's redaction covers telemetry and error traces only; consumer products (ChatGPT, Gemini) expose user-level toggles and deletion but no organizational policy surface [18, 9]; Letta and LangGraph delegate governance entirely to the application. Deletion contracts are similarly absent—the finding formalized in our companion study, which observed one vendor documenting

that derived facts survive episode deletion unless separately invalidated [4]. MLPal Memory treats these as pipeline stages (C1) with drop reasons in telemetry, and its purge path is an auditable governance event.

5.4 Agent integration surface

All surveyed systems now ship MCP servers or equivalents. Their security postures differ sharply. Cognee's MCP server registered 11 tools at v1.4.0 including destructive operations (`delete`, `prune`) with no authentication on the transport, single-user identity, dataset selection keyed to the self-reported client name, and a documented arbitrary-code-execution path via its custom-model loading parameter—the code comments themselves direct operators to impose auth externally. (By v1.5.3 the surface narrowed to four tools with destructive operations internal-only except a full-wipe flag; transport auth remains absent.) MLPal Memory's agent surface is read-only by contract (pinned by test), forwards the caller's own credential, and holds no secrets; writes exist only behind the authenticated REST boundary with scope-gated authorization. We consider a least-privilege default the correct posture for a component that will be wired into autonomous agents.

5.5 What surveyed systems do better

The comparison is not one-sided, and we note capabilities MLPal Memory lacks. Cognee's implicit-feedback loop—rating served context and folding an exponentially-weighted `feedback_weight` back onto graph elements—realizes retrospective reflection [23] in production; its write-side engineering (a rollback ledger written ahead of graph/vector writes, reference-counted deletion of shared entities, stale-run recovery) exceeds ours, and its iterative chain-of-thought retrieval demonstrably helps multi-hop questions at a cost we decline on the default path. Letta's git-backed memory gives version semantics and human-inspectable state we currently approximate only in database history. Mem0's minimal per-fact operations yield the lowest write cost of the graph-capable systems. LangGraph's namespace store is far simpler to operate than anything graph-shaped. We are adopting the first two (outcome-weighted ranking computed off the hot path; write-side rollback and reference-counted purge) and defer versioned commit/publish semantics to the lifecycle layer sketched in section 7.

6 Read-Path Cost Analysis

Memory differs from search in its duty cycle: a memory layer is consulted on *every* agent turn, so per-query cost multiplies across an organization's entire agent traffic. We therefore account for the retrieval path analytically, by counting model invocations in the shipped code of each open-source system's default useful configuration (table 2). Analytic counts are implementation facts, not benchmark estimates, and they bound cost and tail latency from below: each on-path LLM call adds at least one model round-trip to every query.

Two structural observations follow from the source reading, independent of any benchmark.

LLM-on-read couples memory cost to model economics. A mode that spends k model calls per query spends them on every turn of every agent; at organizational duty cycles this dominates total system cost and places model tail latency on the interactive path. Systems that win multi-hop benchmarks with iterative LLM retrieval loops are reporting a quality/cost trade, not a free improvement; we expose an equivalent capability only as an explicitly opt-in deep mode (section 7) so that the default contract stays $O(1)$ database work per query.

Retrieval data structures must push down, not project up. The surveyed graph-completion implementation materializes the graph (whole, or seeded by $\sim$500 vector hits) into application memory per query and scores triplets in Python; its lexical leg loads the full chunk corpus into process memory. These designs are correct at demonstration scale and degrade at enterprise scale. MLPal Memory keeps candidate generation inside the database (HNSW with iterative scanning under scope filters; expression-indexed lexical rank; depth-bounded recursive traversal for the graph boost), so per-query work is bounded by index access paths rather than graph size.

Write-side cost is governed by tier routing (C5): deterministic extraction for high-frequency telemetry, model extraction only for content-bearing episodes, with batching and deferral available because capture is

Table 2: Model invocations on the retrieval path (source-derived, July 2026). Embedding calls are excluded (all systems embed the query once per vector leg). Cognee counts are per query for the named mode with defaults; session-enabled deployments add one pre-answer analysis call to every mode.

System / mode	LLM calls	Notes
MLPal Memory search / explain	0	lexical + ANN + RRF + graph boost; deterministic
Graphiti search	0	hybrid retrieval; optional reranker is non-LLM
Mem0 retrieval	0	dense top-k; answer synthesis left to the caller
Cognee CHUNKS	0	vector lookup, verbatim payloads
Cognee RAG_COMPLETION	1	top-k chunks → completion
Cognee GRAPH_COMPLETION	1	triplet search → completion; graph is projected into process memory per query
Cognee ..._COT (flagship)	≈13	$1 + 4$ rounds × (validate + follow-up + completion)
Cognee session pre-analysis	+1	on every answered turn when sessions are enabled (default on)

asynchronous. We do not claim optimal write cost—Mem0's single-call per-fact updates are cheaper [5]—but write amplification is bounded by salience routing rather than by traffic volume, and the expensive tier produces schema-validated, evidence-linked facts rather than free text.

7 Evaluation

We evaluate a complete local deployment of MLPal Memory on one developer's real working corpus, measuring (i) behavioral correctness, (ii) retrieval quality against a grep baseline over the identical corpus, (iii) temporal (staleness) behavior, (iv) read-path latency, and — in a second, preregistered campaign — (v) the read-mode ladder (section 7.10), (vi) longitudinal convergence and forgetting under a changing world (section 7.11), (vii) scale (section 7.12), and (viii) end-to-end usefulness to an unmodified production agent (section 7.13). We report the full measurement trajectory — including the configurations that *lost* to the baseline and the diagnosed causes — because the correction loop is the methodological point: deterministic, stored, reproducible evals caught three retrieval defects that neither unit tests nor demos surfaced.

7.1 Deployment and corpus

The system under test is the production service running locally (Docker: PostgreSQL 18 + pgvector, API + fold worker, read-only MCP sidecar), populated by three collectors from the developer's machine: 21 substantial Claude Code sessions (filtered from 197 main-thread transcripts; 494 subagent sidechains excluded), 158 curated markdown/skill files, and 68 git repositories (113 README/architecture documents plus deterministic repo-card facts) — 292 documents, 4,107 chunks, and 205 typed nodes after the governed fold (consent → policy → secret redaction → extraction → bi-temporal write). Every document carries its own event time (session start, file mtime, or last git commit date), so the corpus contains genuinely stale knowledge — the realistic condition. Ingestion is idempotent end-to-end (content-hashed client ids + server-side dedup), verified by re-running the collectors.

Embedding spaces. The correction trajectory (section 7.3) was measured with a deterministic offline embedder (hashed bag-of-tokens: token-overlap signal, no semantics) — a deliberate lower bound that exercises the fusion layer's embedder-quality input. The final configuration was then re-measured in two real semantic spaces (section 7.4): a local in-process model (bge-small, ONNX — the open-source default, no network or key) and the production gateway's `text-embedding-3-small`. Space migration is a first-class operation: every embedded row is stamped with its space name, and an idempotent re-embed tool moves chunks, entity names, and fact sentences between spaces without mixing them.

Table 3: Retrieval-quality trajectory on the real corpus (20 queries, hit@5/MRR; p95 over the query set). Every row is a stored, reproducible run in `evals/results/`; R5 is retained as the measured regression that was reverted.

Run	Configuration change	hit@5	MRR	p95
R0	initial (phrase-ILIKE lexical leg)	0.20	0.11	167 ms
R1	+ term FTS, AND semantics	0.25	0.15	118 ms
R2	+ OR-semantics coverage ranking	0.35	0.26	190 ms
R3	+ quality-weighted fusion, per-doc diversity	0.40	0.28	250 ms
R4	+ IDF term weighting	0.55	0.39	1188 ms
R5	tf-saturating variant (*regressed; reverted*)	0.45	0.30	3114 ms
R6	+ stored tsvector column, workspace facet	**0.60**	**0.40**	**109 ms**
	grep baseline (identical corpus, file-level)	0.55	0.44	—

7.2 Behavioral correctness

The behavioral suite grew from 160 to 198 offline tests plus 12 Postgres-only tests run against the migrated docker database, covering: cross-tenant and cross-scope isolation (negative), owner-only personal memory including administrators, team-membership gates, scope-confined bounded graph traversal, consent tri-state with purge-on-clear, deny-precedence policy, secret redaction, bi-temporal supersession and as-of reads in both timelines, working-tier TTL (expiry filters independent of sweep timing; promotion on durable re-observation; no downgrade), publish/endorse/contend semantics with contested labeling, dead-letter bounded retries, and the read-only MCP contract. An end-to-end smoke (ingest → fold → search → as-of → purge) runs against the live stack; its first execution caught a real deployment bug (schema-only `search_path` making the `vector` type unresolvable on any fresh install) that CI had structurally missed.

7.3 Retrieval quality: the correction trajectory

Twenty hand-curated queries over the real corpus, each with programmatic gold criteria (parent-document URI fragment or content regex — no LLM judge), scored hit@5 and MRR against a term-frequency grep baseline ranking the *identical* ingested files (table 3).

The final configuration *exceeds* the grep baseline on hit@5 (0.60 vs. 0.55) with complementary coverage: memory uniquely answers the semantically-phrased and cross-session queries (e.g. "what did we find when analyzing X", costs recalled from a months-old session), grep uniquely wins two keyword-heavy lookups, and their union covers 75% — with the offline non-semantic embedder. A workspace-facet ablation isolates the hierarchy's contribution: removing the facet costs 5 points (0.60 → 0.55), and the ablation itself caught a wiring gap (the facet boost initially reached only the derived tier — measured as a zero-effect ablation, then fixed).

7.4 Semantic embeddings and measured baselines

Re-measuring on the grown corpus (344 documents, 5,458 chunks, including the later experiment ingests) with real semantic embeddings (table 4): with the production embedder the full pipeline reaches **0.65 hit@5 / 0.53 MRR**, now exceeding grep on *both* metrics (the offline-embedder configuration lost MRR, which we reported; the remedy was embedding quality, not scoring changes). The single-leg arms are the measured versions of two standard baselines on identical data: the vector leg alone is a classic dense-RAG pipeline, the lexical leg alone an IDF-weighted full-text search.

Two honest readings. First, with a strong embedder *each leg alone* also reaches 0.65 hit@5 on this goldset (their union covers 14/20 vs. the hybrid's 13/20 — the fusion trades one lexical-only hit for one vector-only hit); at n=20, one-hit differences are noise, and we do not claim fusion adds coverage over the best single leg. Fusion's measured value is ranking quality over lexical (MRR 0.53 vs. 0.40) and robustness across embedder quality — the trajectory above shows the same pipeline holding 0.60 when the vector leg carries no semantics at all, which is precisely the regime a self-hosted deployment with a weak or absent embedder

Table 4: Semantic-embedding configurations and single-leg baselines (same 20 queries, identical corpus). The starred p95 values are dominated by a laptop-to-production embedding round-trip (∼0.7–0.9 s); in-cluster this call is single-digit milliseconds, and the local embedder runs in-process.

Configuration	hit@5	MRR	p95
Full hybrid, gateway embeddings	**0.65**	**0.53**	995 ms*
Vector leg only (dense RAG), gateway	0.65	0.53	769 ms*
Lexical leg only (IDF-FTS)	0.65	0.40	62 ms
Full hybrid, no workspace facet	0.60	0.52	455 ms*
Full hybrid, local embedder (OSS default)	**0.65**	**0.54**	**148 ms**
grep baseline (identical corpus)	0.55	0.44	—

occupies. Second, the workspace facet remains worth +5 points in the semantic configuration (0.65 → 0.60 without), consistent with its contribution under the offline embedder. Third, the *local in-process* embedder (bge-small via ONNX — the open-source default, requiring no key and no network) matches the hosted production embedder on this corpus (0.65/0.54 vs. 0.65/0.53) at 148 ms p95 end-to-end: self-hosting this system does not trade away retrieval quality.

Three defects were diagnosed and fixed strictly one-variable-at-a-time: (R1) the direct tier's lexical leg matched the *whole question* as an ILIKE phrase — silent for multi-word queries, leaving only the weak vector leg; (R2) its replacement used `plainto_tsquery`, whose AND semantics require a chunk to contain *every* content word of a question; (R3) rank fusion weighted the self-reportedly non-semantic dev-hash vector leg 1:1 against the lexical leg, diluting it, while top-k slots were crowded by multiple chunks of the same document. The fixes — OR-semantics term queries ranked by coverage, quality-weighted reciprocal-rank fusion, and per-document diversity — are each general mechanisms, not benchmark tuning; the suite pinned each with a regression test.

7.5 Temporal behavior

The staleness property is tested behaviorally: two vintages of the same fact (a 700-day-old "maven" document and a 5-day-old "gradle" document) must invert under recency-decay ranking (half-life 180 days, floored so old knowledge stays reachable), while an `as_of` query bypasses decay entirely and returns the period-true answer. Both hold in CI and against the live corpus, where session evidence spans March–July 2026 and packet *Freshness* sections report source-age ranges.

7.6 Verification probes: contracts run against the deployment

The maintenance contracts are additionally *executed* against the live deployment — not only asserted in CI — by a probe suite that operates through the public API under an isolated identity and emits machine-readable verdicts. The **freshness probe** (C4a) ingests two vintages of one fact (700 and 2 days old) and requires the recency inversion plus source-age reporting in the served packet. The **deletion probe** (D1) ingests a uniquely-markered document, verifies retrievability, issues the consent-CLEAR purge, and then verifies every read surface clean — emitting a *deletion certificate* listing the purge counts (documents, chunks, derived nodes and edges) and each surface checked. The **rebuild probe** (C1) replays the collectors twice over the identical corpus with client state forgotten and requires every unchanged input in run two to be recognized by content identity with store counts identical. Its live result: 317 of 318 documents recognized as duplicates; the single "new" document was the transcript of the session *running the probe*, which grew between the two reads — correctly detected drift, distinguished from a determinism violation by source modification time.

The deletion probe's first live run caught a real defect — the second deployment-grade bug found by execution-based verification in this project, after the smoke test's `search_path` find: session commits ran in the framework dependency's teardown, *after* the response is sent, so two sequential governance writes could commit out of order. The observed instance left a scope opted out that its owner had just re-armed — precisely the class of silent governance failure the contracts exist to exclude. The fix commits governance mutations in the handler, making the decision durable and ordered the moment the caller observes success;

the probe suite then passed three consecutive runs. We take this as the methodological claim in miniature: correctness properties of a governed store must be exercised against the deployed system, where framework buffering, transaction ordering, and configuration exist, not only in unit scope.

7.7 Latency

The deterministic read path holds its budget on the live stack: p95 109 ms cold and 71 ms warm over the query set against 4,107 chunks (budget: 300 ms), after a measured excursion to 2.8 s when IDF document frequencies were computed by recomputing `to_tsvector` per candidate row — fixed with a stored generated tsvector column (computed once per write) and a per-tenant document-frequency cache. No retrieval mode issues a model call; the memory-packet endpoint (`/memory/answer`) assembles facts, evidence, contested labels, gaps, and freshness deterministically (105 ms observed end-to-end in the UI).

7.8 Agent-in-the-loop A/B

The behavioral claim was tested end-to-end (2026-08-24): same harness (yodex 0.8.0), same pinned model, same tasks, isolated per-arm environments; the only variable is the memory package (MCP configuration + a usage instruction). Four tasks re-introduce defects this organization previously found and fixed — the fix experience exists in memory as ingested sessions (*regression reintroduction*: the design measures recall-and-apply, which is the product claim, not general problem-solving uplift) — plus two memory-irrelevant guardrail tasks bounding negative transfer. Hidden verification suites were two-way validated. N=3 per cell, 36 runs.

Result, reported straight: a null on efficiency at this task scale. Both arms pass 18/18. Over the memory-relevant tasks, wall time differs by −0.7%, output tokens +1%, cache-read tokens +3.4%, tool calls +1% — within run-to-run noise. Guardrails show no negative transfer. Transcript forensics explain the mechanism: the memory arm consulted memory within the first seconds with precisely the right query, and the packet delivered the root cause immediately — the agent's next message begins "The memory already flagged a past incident about this exact pattern…". It then spent the same time as baseline on implementation, the prompt-mandated regression test, and verification. *Memory eliminated the diagnosis phase; on a small, single-repo task with a frontier model, diagnosis was never the bottleneck.* The mechanism the system claims — recall in seconds of what the organization already learned — worked in every memory-arm run; the task scale made the saved phase too small to move totals. We report this null in full because our own methodological position demands it, and it sharpens the next experiments: exploration-dominated tasks (large, unfamiliar, cross-repo), a capability-limited model arm where diagnosis-in-hand may move pass rate rather than time, injection-vs-tool delivery, and time-to-first-correct-edit as the primary metric.

7.9 Escalation study: knowledge vs. capability

A second campaign probed the exploration-dominated regime on a 794k-LOC SWE-bench instance the harness had historically lost to budget exhaustion, under an "attempt-2-with-memory-of-attempt-1" protocol (turn budget pinned; graders two-way validated). The chain of results, each fix shipped as a product mechanism before the next arm ran: (i) raw-transcript memory performed identically to a cold retry — similarity retrieval over a failed exploration returns the problem statement (distillation is necessary); (ii) distilled memory collapsed time-to-first-edit $328s \rightarrow 53s$ — but transmitted the attempts' plausible-but-wrong root-cause theory at the same speed (outcome provenance is necessary); (iii) with verified facts leading the packet, the agent followed the failed attempts' confident *narrative* in the evidence tier (provenance must cover every tier reaching the context window; failed-run evidence is now inline-labeled and downranked); (iv) with the packet injected into the system prompt, the agent quoted the failed hypothesis from the do-not-follow section while skipping the verified fact — models mine memory for prior-confirmation, so agent-mode packets now *suppress* failed-run narrative entirely, rendering negative knowledge as one-line constraints.

The terminal result draws the boundary: the frontier-model arm solved the task unaided (1/1, editing the gold-patch file); the mid-tier arm failed under *every* memory configuration (0/14) — including packets naming the exact fix location and ruling out the wrong path. **Memory transmits knowledge, not capability.** The economic claim this licenses is scoped accordingly: a frontier diagnosis paid once becomes

cheap-tier work *when the downstream task is knowledge-limited rather than capability-limited* — a distinction measurable per task type through the tuning ledger, and itself a routing signal.

7.10 The answer ladder: deciding read modes by measurement

A dedicated campaign (goldset grown 20 → 40 queries, groundedness graded against retrieved evidence) decided the read-path product shape empirically rather than by taste. Three modes competed: the deterministic packet (zero model calls), one-shot synthesis over the packet (one call), and a bounded *memory hop* — a retrieve–reformulate loop that re-queries with new formulations until it can answer or a hop budget is exhausted. The one-shot synthesizer *lost* to the free packet on groundedness (48% vs. 57%); its measured value is presentation, not accuracy — so it is priced and labeled as such. The hop reached **72% grounded** with 100% abstention correctness — the only arm above the one-shot ceiling (+15pp, six queries, beyond the noise floor of this goldset; the preregistered 75%-answered target was met). Two mechanisms made the loop safe to ship: server-side citation enforcement (any citation not actually retrieved is stripped and counted — zero invented citations survive to the caller across all runs) and an early-stop rule (a hop whose retrievals surface no new citations terminates the loop — reformulation cannot help). A write-side fix from the same campaign (recency-first document admission with per-repo budgets) lifted the free packet itself from 57% to 62%. The shipped ladder follows the measurements: deterministic packet as the free default, hop as the priced deep route, synthesis as presentation only.

The same loop was then run by an unmodified external agent against live infrastructure questions (twelve tasks over a real Kubernetes estate, mutations forbidden): generation-0 scored 2–6/12 and fell into a stale-context trap; with the memory serving current-truth packets, generation-1 scored 7/12, avoided the trap, and issued zero mutations. The residual failure class — mixing facts from different eras when synthesizing a policy — occurs at the composition level, not retrieval, and motivates the as-of-now packet variant left as future work.

7.11 Longitudinal replay: does memory track a changing world?

Bi-temporal machinery is only useful if the store actually converges on the new truth as a corpus evolves. We replayed one real five-week operational corpus (a cloud-account migration: costs, versions, and decisions all changed mid-stream) in chronological checkpoints, grading a fixed question set at each point with era-aware golds. Timeline accuracy rose monotonically (67% → 92% across checkpoints), 10 of 12 changed facts were picked up within 0–2 days of their flip, and *graceful forgetting* held: exactly one stale-era answer (steady, not growing) with nothing deleted — superseded values remain reconstructable, and as-of reconstruction agreed with the recorded history at 99%. The replay also isolated a failure class the mechanism-level tests missed: scalar values (a daily cost, a cluster version) merged into generic entities and drifted. The fix — typed value-facts with stable keys, same-key supersession, and a two-tier extractor (guarded patterns plus a cost-gated LLM precision tier with verbatim-quote validation) — closed the class: on the final replay the late checkpoints graded 100% with clean value histories.

7.12 Scale

The single-store bet was stress-tested by growing one tenant synthetically from 5k to 1,000,000 chunks on one `r6i.2xlarge` (total experiment cost ≈ $2): probe-set hit@5 held at 100% throughout and read latency stayed flat at 142 ms p50 at 1M chunks. Latency excursions during the runs traced to concurrent *write* contention, not corpus size — capacity planning should watch fold throughput, not chunk count.

7.13 Preregistered usefulness ablation: an agent with and without memory

The launch-gating question is not retrieval quality but downstream usefulness: does an unmodified production agent do better work with the memory attached? We preregistered a ten-task study (tasks fixed before any run) over the real org corpus: Claude Code, identical model, identical working directory (the full code root — the baseline may grep everything), identical tool permissions; the only difference is the memory

Table 5: Usefulness ablation (10 preregistered org questions, machine-checked; medians over tasks). *one task regraded post-hoc with a loosened check applied identically to both arms; original grading (9/10, 2/3) preserved in the released results.

	baseline agent	agent + memory
correct	6/10	**10/10***
staleness-trap subgroup	0/3	**3/3***
median wall time	15.1 s	21.8 s
median cost/task	$0.0271	$0.0278
median agent turns	3	3

MCP and one sentence of tool guidance. Three tasks are *staleness traps*: the true answer changed over time and outdated statements outnumber current ones on disk.

Accuracy is effectively free at this scale: cost per task (computed from token telemetry at list rates) differs by under $0.001 and turn counts are identical; the wall-time cost is one MCP round-trip at the median (table 5). The staleness subgroup is the finding: on all three changed-world questions the baseline did not confabulate — it *bounced the question back to the human* ("remind me of the decision and timeline"). Without a memory layer, the human is the memory; the ablation prices exactly that. Unlike the earlier A/B (section 7.9), this design changes *only* memory availability — instructions and tool surface are otherwise identical — so the treatment is not bundled. The harness, preregistration, and raw outputs are released; the study is mechanism-scale (n=10 tasks, one run per arm) and its tasks were authored by the system's builders before the runs — the replication kit is the harness itself, runnable on any org's corpus.

7.14 Same-corpus head-to-head: Mem0

The comparison in section 5 is source-grounded but structural; one competitor admits a same-test measurement. We preregistered a head-to-head against Mem0's open-source release (v2.0.19): the identical 60-document operational corpus ingested into both systems (Mem0 in three configurations — its default LLM extraction pipeline, raw storage, and raw with hybrid BM25 — so no configuration argument survives), the twelve changed-world questions from section 7.11, and an identical reader model answering from each system's default read. Final scores: our packet 11/12 correct with one outdated-value answer; Mem0's best arm 10/12 with two — a one-question margin at n=12, which we state as such. The memory-layer result is the cleaner one: our store served the current truth on 12/12 (current value plus labeled history), while Mem0's own store, audited directly, never contained the current cluster version in any configuration — its default pipeline distilled the 0.8M-character corpus to twenty memories, and an ADD-only write path cannot recover a missed update at read time. Mem0 offers no as-of read. We publish our losing arms (9/12 before two disclosed retunes): Mem0's distillation initially beat our retrieval on state flips announced as passing mentions in dense logs, and that loss produced a mechanism — lifecycle state-facts on the watched-fact machinery (closed-enum extraction, verbatim-quote validation, deterministic subject canonicalization, same-key supersession, and a keyed read-time lookup) — built, test-pinned, and re-measured before this section was written.

7.15 Limitations

Retrieval quality is measured on one developer's corpus with author-curated golds (n=40 after the ladder campaign — still too small to resolve one-hit differences between strong configurations), and the corpus includes sessions about building the system itself; the earlier A/B (section 7.9) bundles instruction and tools, decomposed only by the later ablation (section 7.13), which is itself mechanism-scale (n=10, one run per arm, builder-authored tasks); the hop's early-stop rule ships as a partial mitigation (it terminates no-progress loops but does not bound worst-case hop cost tightly); and org-scale multi-writer dynamics (contention, authority ranking) are designed and tested behaviorally but not yet measured under load. These are the next measurements, not open designs.

8 Conclusion

The memory literature has optimized what to remember and how to rank it; the systems beneath those algorithms have been under-specified. MLPal Memory takes the position that for enterprise agents the binding constraints are elsewhere: memory must respect trust boundaries between individuals and organizations, enforce governance before persistence, keep a bi-temporal record that can be audited and corrected, tie every derived belief to verbatim evidence, and answer on every agent turn without spending model inference to do so. We showed these commitments are jointly realizable in a small system on boring infrastructure—one relational store, one governed pipeline, one audited isolation predicate—and, through source-grounded comparison, that no contemporary system combines them: the field's most temporally rigorous engine has no governance surface, its most governed engine has flat tenancy and an LLM-loaded read path, and none models the subject hierarchy organizational knowledge actually has.

Two companion papers supply the accountability half of the program: an instrument that measures what memory systems destroy at write time and whether their maintenance claims hold [4], and a theory of sound selective repair that makes correction affordable [3]. This paper supplies the system those instruments were built to hold to account. The open problems we consider most consequential—reviewable publication of personal knowledge into shared scopes, principled ranking of persistent human disagreement, and governed export of memory as training data—are, we argue, where agent memory research should go next: not because recall quality is finished, but because memory that cannot be governed, audited, or afforded will not be deployed to find out.

References

[1] B. Thomas Adler and Luca de Alfaro. A content-driven reputation system for the Wikipedia. In *Proceedings of the 16th International Conference on World Wide Web (WWW)*, 2007.

[2] Amazon Web Services. Amazon bedrock AgentCore memory. `https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/memory.html`, 2026. Accessed July 2026.

[3] Anonymous. Captured, not declared: Sound selective repair for LLM memory pipelines. *Under review*, 2026. Companion paper.

[4] Anonymous. Total recall, provable forgetting: Measuring fidelity and maintenance contracts of LLM memory over multi-source work data. *Under submission, PVLDB*, 2026. Companion paper.

[5] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

[6] Cognee. Knowledge graph memory benchmarks: A deep dive. `https://www.cognee.ai/blog/deep-dives/knowledge-graph-memory-benchmarks`, 2026. Accessed July 2026.

[7] Community reproductions. On disputed LoCoMo reproductions across memory vendors. Public vendor exchanges: Zep 84% vs. Mem0 reproduction 58.4% vs. Zep counter 75.1% on identical suites, 2026. Accessed July 2026.

[8] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of SIGIR*, 2009.

[9] Google. Gemini apps personalization. `https://support.google.com/gemini/answer/16598469`, 2026. Accessed July 2026.

[10] Luke Guerdan, Solon Barocas, Kenneth Holstein, Hanna Wallach, Zhiwei Steven Wu, and Alexandra Chouldechova. Validating LLM-as-a-judge systems under rating indeterminacy. In *Advances in Neural Information Processing Systems*, 2025.

[11] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems*, 2024.

[12] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. From RAG to memory: Non-parametric continual learning for large language models. *arXiv preprint arXiv:2502.14802*, 2025.

[13] Yufang Hou, Alessandra Pascale, Javier Carnerero-Cano, Tigran Tchrakian, Radu Marinescu, Elizabeth Daly, Inkit Padhi, and Prasanna Sattigeri. WikiContradict: A benchmark for evaluating LLMs on real-world knowledge conflicts from Wikipedia. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024.

[14] Andrew Kane. pgvector: Open-source vector similarity search for Postgres. `https://github.com/pgvector/pgvector`, 2026. Accessed July 2026.

[15] LangChain. LangGraph memory: Concepts. `https://docs.langchain.com/oss/python/concepts/memory`, 2026. Accessed July 2026.

[16] Letta. Context repositories: Git-based memory for AI agents. `https://www.letta.com/blog/context-repositories/`, 2026. Accessed July 2026.

[17] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. *arXiv preprint arXiv:2402.17753*, 2024.

[18] OpenAI. Memory FAQ. `https://help.openai.com/articles/8590148-memory-faq`, 2026. Accessed July 2026.

[19] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[20] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[21] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

[22] Haoran Tan, Zeyu Guo, Zhengyu Shi, Lu Li, Zijun Yin, Rui Sun, Xiao Zhao, et al. MemBench: Towards more comprehensive evaluation on the memory of LLM-based agents. *arXiv preprint arXiv:2506.21605*, 2025.

[23] Zhen Tan, Jun Yan, I-Hung Hsu, Rujun Han, Zifeng Wang, Long T. Le, Yiwen Song, Yanfei Chen, Hamid Palangi, George Lee, Anand Iyer, Tianlong Chen, Huan Liu, Chen-Yu Lee, and Tomas Pfister. In prospect and retrospect: Reflective memory management for long-term personalized dialogue agents. *arXiv preprint arXiv:2503.08026*, 2025.

[24] TerminusDB. Knowledge graph version control. `https://terminusdb.org/docs/knowledge-graph-version-control/`, 2026. Accessed July 2026.

[25] Topoteretes. Cognee: Memory for AI agents in 6 lines of code. `https://github.com/topoteretes/cognee`, 2026. Version 1.4.0, accessed July 2026.

[26] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.

[27] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.

[28] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018.

[29] Zep Inc. Graphiti: Build real-time knowledge graphs for AI agents. `https://github.com/getzep/graphiti`, 2026. Accessed July 2026.